Unconditional Statements

Simple calculator with switch-case

- In C, an unconditional statement is a statement that is executed without any condition or consideration of whether a particular condition is true or false.
- Unconditional statements are executed sequentially, one after the other, and they do not depend on any conditional logic.

3 types

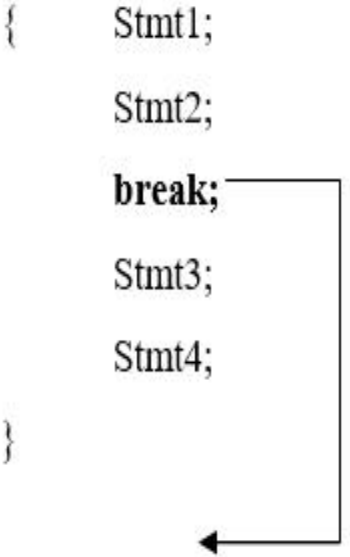
1. break
2. continue
3. goto
4. return

‘Break’ statement

1. Break Statement:

- It is a keyword which is used to terminate the loop (or) exit from the block.
- It is executed unconditionally when encountered.
- The control jumps to next statement after the loop (or) block.
- break is used with for, while, do-while and switch statement.
- When break is used in nested loops then, only the innermost loop is terminated.
- The **break** is a keyword so it must be used only in lower case letters.
- The **break** statement can not be used with **if** statement.
- The **break** statement can be used only in switch case and looping statements.
- The **break** statement can be used with **if** statement, only if that **if statement** is written inside the switch case or looping statements.

Syntax

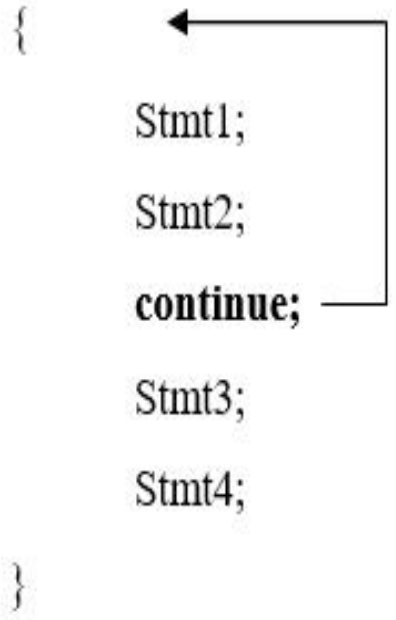


‘continue’ statement

2. Continue Statement: In a loop, the ‘continue’ statement is used to skip the current iteration and move to the next iteration.

- It is executed unconditionally when encountered.
- The **continue** is a keyword so it must be used only in lower case letters.
- The **continue** statement is used only within **looping statements**.
- The **continue** statement can be used with **if** statement, only if that **if statement** is written inside the looping statements.

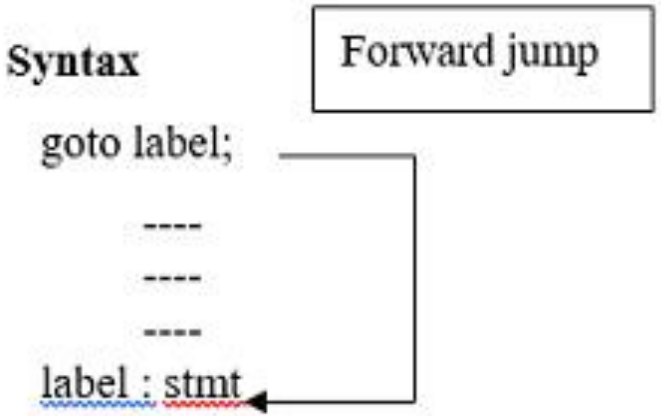
Syntax

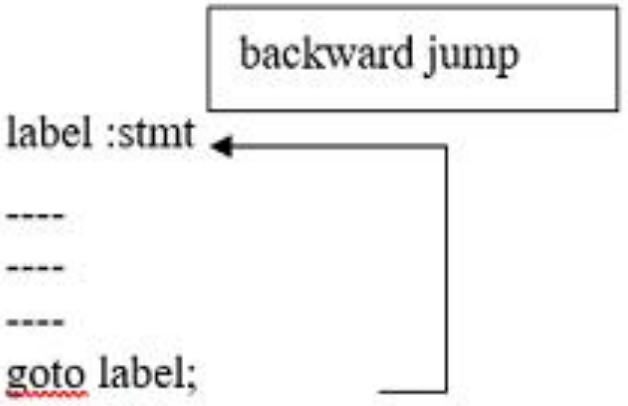


‘goto’ statement

3. goto Statement:-It is used to jump the control from one statement to another statement.

- It is used after the normal sequence of program execution by transferring the control to some other part of program.
- The **goto** is a keyword so it must be used only in lower case letters.
- The **goto** statement must require a **label**.
- The **goto** statement can be used with any statement like if, switch, while, do-while, and for, etc.





```
#include<stdio.h>
void main( )
{
    printf("Hello");
    goto l1;
    printf("How are");
    l1: printf("you");
}
```

```
#include <stdio.h>
int main()
{
  int a = 5, b = 2;
  int ans;
  // Check if b is not zero to avoid division by zero
  if (b != 0)
  {
        ans = a / b;
        // if a is perfectly divisible by b, just print it
        if (a % b == 0)
              goto print_line;
// else 1 needs to be added to the answer, for ceil division
        ans += 1;
}
else
{
      // Handle division by zero error
      printf("Error: Division by zero.\n");
      return 1;
// Non-zero return value to indicate error
}
// defined label
    print_line:
    printf("%d", ans);
    return 0;
}
```

```
#include <stdio.h> //backward jump
void main()
 {
    int start = 1, end = 10;
    int curr = start;
    print_line:
    printf("%d ", curr);
    if (curr < end)
    {
        curr++;
        goto print_line;
     }
     return 0;
}
```

```
#include <stdio.h>
void main()
{
  int num,i=1;
  printf("Enter the number whose table you want to print?");
  scanf("%d",&num);
  table:
  printf("%d x %d = %d\n",num,i,num*i);
  i++;
  if(i<=10)
        goto table;
}
```

‘return’ statement

4. return

- It terminates the execution of function and returns the control of calling function

  Syntax: ***return[expression/value];***
- Returns the flow of the execution to the function from where it is called.
- This statement does not mandatorily need any conditional statements.
- As soon as the statement is executed, the flow of the program stops immediately and returns the control from where it was called.
- The return statement may or may not return anything for a void function, but for a non-void function, a return value must be returned.